

Captura de osciloscopio Siglent por USB

Manual de uso para proyectos de laboratorio

Herramienta de propósito general · Licenciatura en Física

Dirección de Docencia Experimental — Facultad de Ciencias — Universidad de Chile

1 Objetivo

Este manual explica cómo usar `capturar_osciloscopio.py`, una herramienta que descarga una traza (voltaje en función del tiempo) desde un osciloscopio Siglent conectado por USB y la guarda en un archivo CSV simple, listo para analizar con Python, MATLAB, Excel o lo que uses en tu informe.

¿Para qué proyectos sirve?

Para cualquier experimento donde midas una señal transitoria con el osciloscopio y necesites los datos crudos: carga/descarga de un capacitor, respuesta de un sensor óptico, una señal de un acelerómetro, un pulso único de cualquier tipo. El script no asume nada sobre el origen físico de la señal — solo descarga lo que el osciloscopio tenga en pantalla.

¿Todavía no tienes el osciloscopio a mano?

Prueba primero el modo `-demo`: simula el instrumento con una señal sintética, sin hardware ni VISA instalado, para practicar el flujo completo antes del día de laboratorio.

```
python capturar_osciloscopio.py --demo
python capturar_osciloscopio.py --demo --streaming
```

La consola avisa explícitamente que los datos son sintéticos, para que nunca se confundan con una medición real.

2 Alcance

- Soporta osciloscopios **Siglent serie SDS1000(CNL+)** (probado en un SDS1102CNL+) que hablan el protocolo SCPI sobre USBTMC.
- Captura **un canal a la vez** (C1, C2, etc.).
- Pensado para disparo único (SINGLE); también puede leer una traza ya detenida en pantalla.
- `-demo`: simula el instrumento con una señal sintética, sin hardware ni VISA, para practicar.
- No soporta otras marcas de osciloscopio (Tektronix, Rigol, Keysight) ni protocolos distintos de SCPI/USBTMC.

3 Requisitos

3.1 Hardware

- Osciloscopio Siglent serie SDS1000(CNL+).
- Cable USB tipo B (el que normalmente trae el instrumento).
- Un computador con Windows, macOS o Linux — los tres funcionan; los pasos de instalación difieren según cuál uses (sección siguiente).

3.2 Software

- Python 3.9 o superior.
- Paquetes: numpy, pyvisa, pyvisa-py, pyusb (listados en `requirements.txt`).
- En Windows o macOS, además: NI-VISA o Keysight IO Libraries Suite (sección siguiente).

4 Instalación

El script usa el VISA del sistema si está instalado, o cae automáticamente al backend puro de Python (`pyvisa-py`) si no hay ninguno — el caso típico en Linux. Por eso los pasos de acceso al instrumento difieren según tu sistema operativo.

4.1 Entorno de Python (todos los sistemas)

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

En Windows (cmd o PowerShell), el segundo comando es `venv\Scripts\activate` en vez de `source venv/bin/activate`.

4.2 Acceso al instrumento: Windows / macOS

Instala **NI-VISA** (gratis, de National Instruments) o, alternativamente, **Keysight IO Libraries Suite**. El instalador deja todo configurado: controlador USB, permisos, reconocimiento del instrumento.

Sin pasos adicionales

A diferencia de Linux, no hace falta ninguna regla de permisos: una vez instalado NI-VISA, `pyvisa.ResourceManager()` encuentra el instrumento automáticamente.

4.3 Acceso al instrumento: Linux

Linux no permite acceder a dispositivos USB sin privilegios de administrador a menos que exista una regla que lo autorice explícitamente.

```
sudo cp 99-usbtmc-siglent.rules /etc/udev/rules.d/
sudo groupadd -f usbtmc
sudo usermod -aG usbtmc "$USER"
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Paso que se olvida con frecuencia. Después de ejecutar lo anterior, *cierra sesión y vuelve a entrar* (o reinicia) para que el cambio de grupo tenga efecto, y *desconecta y reconecta* el cable USB del osciloscopio. Si no haces ambas cosas, seguirás viendo error de permisos aunque la regla esté instalada correctamente.

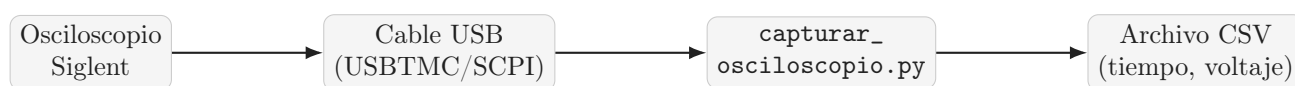
4.4 Verificación (todos los sistemas)

Con el osciloscopio encendido y conectado:

```
python capturar_osciloscopio.py --help
```

Si aparece el mensaje de ayuda sin errores, la instalación está completa.

5 Cómo funciona



El script: (1) se conecta al instrumento, (2) arma un disparo **SINGLE** y espera a que el osciloscopio capture, (3) le pregunta la escala vertical y horizontal que está usando, (4) descarga los puntos crudos de la pantalla y los convierte a voltios y segundos usando esa escala, y (5) los guarda en el CSV.

6 Uso

Comando	Canal	Archivo de salida
<code>python capturar_osciloscopio.py</code>	C1 (por defecto)	<code>traza_real.csv</code> (por defecto)
<code>python capturar_osciloscopio.py C2 datos.csv</code>	C2	<code>datos.csv</code>
<code>... -sin-disparo</code>	el que esté activo	no rearma el disparo
<code>... -streaming</code>	el indicado	guarda solo la <i>última</i> traza

Salida esperada en la terminal:

```

Conectado: Siglent Technologies,SDS1102CNL+,...
20480 puntos capturados en C1.
Ventana: -20.48 us a 184.30 us
Guardado en: traza_real.csv
  
```

6.1 Formato del archivo de salida

Un CSV de dos columnas, con encabezado:

```

tiempo[s],voltaje[V]
-2.048000000e-05,-1.063000000e+00
-2.047000000e-05,-1.061000000e+00
  
```

El instante $t = 0$ corresponde al *trigger* del osciloscopio, no necesariamente al inicio de tu fenómeno físico — eso depende de cómo dispares tu propio experimento.

Dato de muestra

En la carpeta `ejemplos/` hay un archivo `traza_ejemplo.csv` con datos **sintéticos** (no una medición real), solo para que veas el formato sin necesitar el instrumento a mano.

6.2 Captura en vivo (-streaming)

```
python capturar_osciloscopio.py --streaming
```

Abre una ventana con un gráfico que se actualiza con cada captura nueva (arma disparo → espera → descarga → grafica → repite), más un panel lateral con estadísticas de la traza actual (mín./máx., pico a pico, RMS, número de puntos, ventana de tiempo, frecuencia de actualización). Los ejes no se reencuadran en cada captura — el rango se fija con la primera y solo se *expande* si una traza nueva no entra, para que la ventana se vea estable y no salte de escala todo el tiempo. Presiona **Ctrl+C** en la terminal para detener; al salir, guarda la **última** traza en el archivo de salida.

Qué es y qué no es esto. El osciloscopio se comunica por SCPI/USBTMC, un protocolo de pregunta-respuesta (“dame lo que hay en pantalla ahora”), no un canal de streaming continuo del ADC. Cada actualización del gráfico es una captura **SINGLE** completa, repetida tan rápido como el osciloscopio y el USB lo permitan — en la práctica, pocos Hz. Si tu fenómeno se repite más rápido que eso, vas a perderte disparos entre una descarga y la siguiente: no hay buffer de eventos intermedios. Este modo sirve para *monitorear* mientras ajustas el experimento, no para no perderte ningún evento de una serie rápida.

Requiere `matplotlib` (no es necesario para el resto del script).

6.3 Espectro de frecuencias (-fft)

```
python capturar_osciloscopio.py --streaming --fft
```

Agrega un segundo panel con el espectro de amplitud (FFT) de cada captura, en escala log-log. Útil para diagnosticar **ruido de alta frecuencia**: interferencia de fuentes conmutadas, RF, o el propio contenido espectral del transitorio (p. ej. la frecuencia natural de un RLC).

Lo que NO sirve para ver: ruido de red (50/60 Hz). La resolución en frecuencia es $1/\text{ventana}$ — con una ventana típica de pocos cientos de microsegundos, eso son varios **kHz** por punto, demasiado grueso para resolver una frecuencia de 50 Hz. Se necesitaría una ventana de captura mucho más larga (decenas de milisegundos como mínimo), que este script no controla directamente — depende de la base de tiempo configurada en el osciloscopio.

7 Promediar capturas (-promediar)

```
python capturar_osciloscopio.py --promediar 20 promedio.csv
```

Captura N trazas y guarda su **promedio punto a punto**, con la desviación estándar como tercera columna (`incertidumbre[V]`) — el ruido aleatorio cae como $1/\sqrt{N}$ al promediar, el resultado clásico de mediciones repetidas. El resumen en consola incluye la reducción de ruido esperada para el N elegido.

No se combina con `--streaming`

Son dos modos distintos: uno repite y monitorea, el otro repite y promedia. Si se piden ambos a la vez, el script se detiene con un mensaje de error claro en vez de hacer algo ambiguo.

8 Log de estadísticas (-log-estadisticas)

```
python capturar_osciloscopio.py --log-estadisticas sesion.csv
python capturar_osciloscopio.py --streaming --log-estadisticas sesion.csv
```

Agrega una fila (`disparo`, `timestamp`, `V_min`, `V_max`, `V_pp`, `V_rms`, `V_med`) por cada captura al archivo indicado. Si el archivo ya existe, sigue agregando filas — es un log que **crece entre sesiones**, no se sobrescribe. Con `--streaming` se agrega una fila por disparo; sin él, una fila por ejecución del script.

Útil para responder, con tus propios datos y herramientas, preguntas como “¿qué tan repetible fue mi medición?” o “¿mi montaje derivó durante la sesión?” — el script solo registra los números, no decide qué significan.

9 Solución de problemas comunes

Síntoma	Causa probable	Qué hacer
<code>RuntimeError</code> : no se encontró ningún instrumento	Cable USB desconectado o instrumento apagado	Verifica el cable y que el osciloscopio esté encendido
<code>PermissionError</code> al conectar (Linux)	Falta la regla <code>udev</code> , o no reconectaste el cable después de instalarla	Repite la sección <i>Acceso al instrumento: Linux</i> ; reconecta el cable
<code>RuntimeError</code> sin instrumento (Windows/macOS)	NI-VISA no está instalado	Instala NI-VISA o Keysight IO Libraries Suite
<code>TimeoutError</code> : no se detectó disparo	El trigger del osciloscopio no se está cumpliendo	Revisa el nivel y la fuente de trigger en el panel del instrumento
<code>ValueError</code> : longitud de datos inconsistente	Interferencia o corte en la transferencia USB	Vuelve a ejecutar la captura; si persiste, prueba otro cable USB
<code>ImportError</code> al usar <code>-streaming</code>	<code>matplotlib</code> no está instalado	<code>pip install matplotlib</code>

10 Cómo adaptarlo a tu experimento

- **Dos canales:** llama a la función de captura una vez por canal (`C1`, luego `C2`) y junta ambos resultados en un CSV de tres columnas.
- **Varias repeticiones:** usa `-promediar N` si quieres el promedio con incertidumbre. Si en cambio quieres conservar cada traza individual sin promediar, guarda cada una con un nombre distinto (`traza_001.csv`, `traza_002.csv`, ...).
- **Usarlo desde tu propio script:** puedes importar las funciones (`conectar`, `capturar_canal`, `guardar_csv`) directamente en vez de usar la línea de comandos, si tu análisis ya vive en un notebook.

11 Limitaciones actuales

- ☐ Un solo canal por llamada.
- ☐ Solo probado en Siglent SDS1000CNL+; otros modelos Siglent deberían funcionar si comparten el protocolo, pero no está verificado.
- ☐ Sin reintento automático si el USB se desconecta a mitad de una captura.
- ☐ El CSV de salida no guarda la escala ni el modelo del instrumento usados — solo los puntos ya convertidos.
- ☐ `-streaming` no es streaming continuo del ADC: son capturas **SINGLE** sucesivas a pocos Hz, y solo la última se guarda al salir (ver sección *Captura en vivo*).

12 Antes de capturar: lista de verificación

- ☐ El osciloscopio está encendido y el cable USB conectado.
- ☐ La señal que quiero medir ya se ve correctamente en pantalla.
- ☐ El trigger está configurado (nivel y fuente) para disparar con mi señal.
- ☐ El entorno virtual de Python está activado (`source venv/bin/activate`).

□ Sé en qué canal está mi señal (**C1**, **C2**, ...).